

prep/ — von der Rohquelle zum Analysedatensatz

Vier Dateien führen sieben offene Datenquellen zu zwei Panels zusammen.
Alles, was danach kommt, liest nur noch — es rechnet nichts neu.

BACHELORARBEIT

SFFD-Einsatzprognose

PHASE

CRISP-DM 2 und 3

ERGEBNIS

regression.parquet · klassifikation.parquet

Was dieser Ordner leistet

Er beantwortet eine einzige Frage: Welche Zahl darf zu welchem Zeitpunkt im Modell stehen?

QUELLEN

7 offene Datensätze

SFFD-Einsätze, ACS, SFPD (2 Quellen), Land Use, Stadtteilgeometrien, Crosswalk



SCHRITT 1

s1_daten.py

Laden, bereinigen, räumlich und zeitlich verknüpfen



ZWISCHENSTAND

einsaetze.parquet

eine Zeile je Einsatz, mit allen Stadtteilmerkmalen



SCHRITT 2

s2_datensaetze.py

verdichten auf Stadtteil × Monat, Ziele und Folds

DIE KERNIDEE

- Modelliert wird nicht der einzelne Einsatz, sondern der **Stadtteil-Monat** — 35 Stadtteile × 132 Monate.
- Jedes Merkmal muss zum Prognosezeitpunkt bereits veröffentlicht gewesen sein.
- Die Aufteilung in Trainings- und Testgruppen wird hier festgeschrieben, nicht erst im Modellskript.

WARUM DAS DIE HEIKELSTE STUFE IST

- Ein Fehler hier ist in keinem Ergebnis mehr sichtbar — er macht die Zahlen nur **besser**, als sie sein dürften.
- Deshalb liegen alle drei Leakage-Sperren in diesem Ordner und nicht in den Modellen.

Die vier Dateien

DATEI	ROLLE	WAS SIE ERZEUGT	ZEILEN
config.py	Alle Festlegungen an einem Ort	nichts — sie wird nur gelesen	252
s1_daten.py	Laden und Verknüpfen	data/processed/einsaetze.parquet	674
s2_datensaetze.py	Verdichten und Zuschneiden	regression.parquet · klassifikation.parquet	459
build.py	Der eine Startbefehl	Konsolenausgabe und Prüfübersicht	106

AUFRUFREIHENFOLGE

python prep/build.py fährt s1 und dann s2. Die Reihenfolge ist zwingend, weil s2 liest, was s1 schreibt. Einzeln startbar sind beide trotzdem — das ist der Zweck der run_* -Funktionen.

Drei Sperren gegen Informationsvorgriff

Jede sitzt an genau einer Stelle im Code. Wer sie sucht, findet sie an der Funktion, die daneben steht.

1 · PUBLIKATIONSVERZUG

Ein Einsatz aus 2023 bekommt **nicht** den ACS-Jahrgang 2023 — der erschien erst Ende 2024.

- gelöst in `acs_snapshot()`
- Konstante `ACS_PUBLIKATIONS_LAG = 1`

2 · RÜCKWÄRTSFENSTER

Der Kriminalitätsindex eines Monats zählt nur Delikte der zwölf Monate **bis zum Vormonat**.

- gelöst in `kriminalitaetsindex()`
- Konstante `CRIME_FENSTER_MONATE = 12`

3 · STADTTEIL - TRENNUNG

Kein Stadtteil steht je zugleich im Training und im Test. Getestet wird auf **unbekannten Stadtteilen**.

- gelöst in `ergaenze_aufteilung()`
- Spalten `fold` und `ist_holdout`

DATEI

config.py

Alle Festlegungen an einem Ort. Enthält keine einzige Funktion — und genau das ist die Absicht.

252

Zeilen

~40

Konstanten

0

Funktionen

Was hier steht — und warum nichts davon rechnet

Die Datei ist ein Vertrag: Wer eine Zahl ändern will, ändert sie hier und nirgendwo sonst.

GRUPPE	BEISPIELE	WIRKUNG
Pfade	PFAD_REGRESSION, RAW_DIR	wo Roh- und Ergebnisdateien liegen
Quellen	ACS_YEARS, DOWNLOAD_*	welche Jahrgänge geladen werden; Schalter gegen unnötige API-Aufrufe
Zeitfenster	START = 201501, ENDE = 202512	der Analysezeitraum: 132 Monate
Sperren	ACS_PUBLIKATIONS_LAG, CRIME_FENSTER_MONATE	die beiden zeitlichen Leakage-Sperren
Merkmale	PRAEDIKTOREN, SAISON, LAGS	die zehn Prädiktoren; Saison als sin/cos; Lags bleiben deskriptiv
Zielgrößen	NFIRS_GRUPPEN, KLASSEN	die Abbildung der NFIRS-Codes auf vier Klassen
Validierung	N_FOLDS = 5	fünf Folds plus ein Hold-out

KOLLOQUIUMSFRAGE

„Warum steht WIEDERHOLUNGEN nicht hier?“ — Weil `config.py` festlegt, was in die Dateien *geschrieben* wird. Die Zahl der Wiederholungen betrifft nur das Rechnen und steht deshalb in `modelle/config_modelle.py`. Dass die Vorprüfung sie von dort holt, ist eine bekannte strukturelle Schwäche.

DATEI

s1_daten.py

Laden und Verknüpfen. Aus sieben Quellen wird eine Tabelle mit einer Zeile je Einsatz.

674

Zeilen

14

Funktionen

2

Einstiegspunkte

Zwei Phasen, zwei Einstiegspunkte

PHASE A — HERUNTERLADEN

`run_download()` holt die Rohdaten in `data/raw/`. Über die `DOWNLOAD_*` - Schalter abschaltbar, damit ein Wiederholungslauf nicht erneut über die APIs geht.

- `_get()` — HTTP mit Wiederholversuchen
- `lade_datasf()` — die DataSF-Quellen
- `lade_acs()` — die Census-Jahrgänge

PHASE B — ZUSAMMENFÜHREN

`run_join()` baut daraus `einsaetze.parquet`. Hier liegen die inhaltlichen Entscheidungen.

- Bereinigen: `prepare_sffd()`
- Sozialstruktur: `acs_*`, `join_acs()`
- Kriminalität: `crime_monatlich()`, `kriminallitaetsindex()`
- Bausubstanz: `land_use_je_neighborhood()`

DIE RÄUMLICHE KLAMMER

Drei Quellen liefern Koordinaten statt Stadtteilnamen. Sie werden über **dieselben** Stadtteilpolygone zugeordnet (`neighborhoods_gdf()`) — sonst bezögen sich Kriminalitäts- und Baumerkmale auf leicht verschiedene Flächen.

Die drei Ladefunktionen

Reine Beschaffung. Sie treffen keine inhaltliche Entscheidung.

`_GET(URL, PARAMS)`

EIN URL und Parameter

AUS Antwort der Anfrage

- kapselt Wiederholversuche bei Zeitüberschreitung
- damit ein Netzaussetzer keinen Ladelauf beendet

`LADE_DATASF(NAME, LIMIT)`

EIN Datensatz-ID der Socrata-API, Zielpfad

AUS Parquet-Datei in data/raw, Zeilenzahl

- paginiert, weil die API je Anfrage deckelt
- setzt die Spaltentypen ausdrücklich — sonst rät pandas bei GEOIDs auf Ganzzahl und die führende Null verschwindet

`LADE_ACS(YEAR)`

EIN Jahrgang und Variablenliste

AUS eine CSV je Jahrgang

- eigene Funktion, weil die Census-API ein anderes Format liefert: Kopfzeile plus verschachtelte Datenliste

KOLLOQUIUMSFRAGE

„Warum stehen die Typen im Code statt automatisch erkannt?“ — Weil ein Census-Tract-Schlüssel wie `06075010100` als Zahl gelesen die führende Null verliert und dann auf keinen Crosswalk-Eintrag mehr passt. Der Join lieferte still ins Leere.

prepare_sffd(df)

Bereinigt die rohe Einsatztabelle des SFFD.

EIN die rohe SFFD-Tabelle

AUS dieselbe Tabelle, bereinigt und um Zeitspalten ergänzt

WAS PASSIERT

- Doppelte Meldungen desselben Einsatzes werden über die Einsatznummer entfernt
- Die Antwortzeit wird aus Melde- und Ankunftszeitstempel berechnet
- Zeitmerkmale werden abgeleitet: Jahr, Monat, `jahr_monat` als Schlüssel
- Stadtteilnamen werden vereinheitlicht

WORAUF ES ANKOMMT

- Die Zahl der entfernten Dubletten wird **ausgegeben**, nicht stillschweigend geschluckt.
- Ohne den Dedup zählte derselbe Einsatz zweimal — und die Zielgröße wäre systematisch zu hoch.

KOLLOQUIUMSFRAGE

„**Woran erkennen Sie eine Dublette?**“ — An der Einsatznummer. Ein Einsatz kann mehrfach gemeldet werden, behält aber dieselbe Nummer. Verschiedene Fahrzeuge zum selben Einsatz sind *keine* Dubletten in dieser Tabelle, weil sie bereits auf Einsatzebene geliefert wird.

acs_je_neighborhood(acs, crosswalk)

Hebt die Sozialdaten von der Tract- auf die Stadtteilebene.

EIN ACS-Tabelle auf Census- Tract-Ebene und der Crosswalk

AUS eine Zeile je Stadtteil und Jahrgang

DAS PROBLEM

Der American Community Survey liefert Census Tracts — kleinere Einheiten als die 35 Stadtteile. Mehrere Tracts gehören zu einem Stadtteil und müssen zusammengefasst werden.

DIE LÖSUNG, UND WARUM SIE ZWEIGETEILT IST

- **Zählgrößen** (Einwohner, Haushalte) werden **summiert**.
- **Mediane** (Einkommen, Miete) werden **bevölkerungsgewichtet gemittelt** — ein Median lässt sich nicht addieren.

KOLLOQUIUMSFRAGE

„Ist der gewichtete Mittelwert von Medianen ein Median?“ — Nein, und das ist eine bewusst in Kauf genommene Näherung. Der echte Stadtteil-Median ließe sich nur aus Mikrodaten berechnen, die der ACS auf dieser Ebene nicht veröffentlicht. Die Näherung gilt für alle Stadtteile gleich und verzerrt den Verfahrensvergleich deshalb nicht.

acs_snapshot(jahr, acs_years)

Die erste Leakage-Sperre: Welcher Jahrgang war damals überhaupt veröffentlicht?

EIN Einsatzjahr und die Liste verfügbarer ACS-Jahrgänge

AUS der Jahrgang, der verwendet werden darf

DIE REGEL

Zulässig ist nur ein Jahrgang mit

$\text{acs_jahr} \leq \text{Einsatzjahr} - \text{ACS_PUBLIKATIONS_LAG}$

Davon der **jüngste**.

ZWEI STUFEN DER ABSICHERUNG

- **Stufe 1:** der letzte verfügbare Jahrgang, nicht der zeitlich nächstgelegene. Der nächstgelegene könnte in der Zukunft liegen.
- **Stufe 2:** zusätzlich die reale Publikationsverzögerung von rund einem Jahr.

WARUM BEIDES NÖTIG IST

Ohne Stufe 2 bekäme ein Einsatz aus 2023 den ACS-Jahrgang 2023 — der aber erst Ende 2024 erschienen ist. Das Modell hätte damit Information benutzt, die zum Prognosezeitpunkt nicht existierte. Vor dem ersten Snapshot gibt es keinen vergangenen Jahrgang; dort wird auf den ältesten zurückgegriffen — als dokumentierte Limitation, die die Hauptanalyse ab 2015 nicht mehr betrifft.

join_acs(sffd, nb_per_year)

Hängt jedem Einsatz die Sozialstruktur seines Stadtteils an.

EIN Einsatztabelle und die ACS-Jahrgänge je Stadtteil

AUS Einsatztabelle mit den fünf sozioökonomischen Merkmalen

WAS VERKNÜPFT WIRD

- Medianes Haushaltseinkommen
- Armutsquote und Akademikerquote
- Mediane Miete und Leerstandsquote
- Gesamtbevölkerung — später Offset, kein Merkmal

DER SCHLÜSSEL

Verknüpft wird über **Stadtteil × zulässigem ACS-Jahrgang**, den `acs_snapshot()` bestimmt hat.

Damit trägt jede Einsatzzeile genau die Sozialdaten, die zu ihrem Zeitpunkt öffentlich waren.

KOLLOQUIUMSFRAGE

„**Wie oft wechselt der Jahrgang?**“ — Die ACS-Jahrgänge sind 2009, 2014, 2019, 2021 und 2023. Innerhalb eines Blocks sind die Sozialmerkmale also über mehrere Jahre konstant. Genau das ist später der Grund, warum die effektive Stichprobe so klein ausfällt.

Die räumliche Zuordnung

Zwei Funktionen, eine gemeinsame Geometrie.

NEIGHBORHOODS_GDF()

EIN nichts

AUS GeoDataFrame der 35 Stadtteilpolygone

- Wird von beiden räumlichen Verknüpfungen benutzt.
- Absicht: Kriminalitäts- und Baumerkmale beziehen sich auf **identische** Flächen.

LAND_USE_JE_NEIGHBORHOOD()

EIN Parzellentabelle und Stadtteilgeometrien

AUS eine Zeile je Stadtteil mit den baulichen Merkmalen

- Jede Parzelle wird über ihren Mittelpunkt einem Stadtteil zugeordnet, dann aggregiert.
- Ergebnis: Altbauanteil vor 1940, Wohngebäudeanteil, Anteil Risikogewerbe.

DIE WICHTIGSTE EINSCHRÄNKUNG DIESER DATEI

Die Baudaten sind ein **Snapshot aus 2020** — der einzige verfügbare Jahrgang. Sie sind damit über den gesamten Analysezeitraum konstant. Zusammen mit den nur alle paar Jahre wechselnden ACS-Daten heißt das: Von zehn Prädiktoren variiert nur einer monatlich.

crime_monatlich(hist, neu, crosswalk)

Zählt Delikte je Stadtteil und Monat aus zwei Polizeiquellen.

EIN die historische SFPD-Tabelle (bis 2017), die moderne (ab 2018-01) und der Crosswalk

AUS eine Zeile je Stadtteil und Monat mit der Deliktzahl

ZWEI QUELLEN, EIN SCHNITT

- Das SFPD hat im Mai 2018 sein Meldesystem gewechselt.
- Die **moderne** Quelle ist voraggregiert und bringt eine Stadtteilspalte mit.
- Die **historische** hat nur Koordinaten — dort ein räumlicher Join ins Polygon.

WARUM DIE KATEGORIEN SUMMIERT WERDEN

Der spätere Index zählt **alle** Straftaten. Damit erübrigt sich eine Harmonisierung der beiden Kategorienschemata — die ohnehin nicht deckungsgleich abbildbar wären und eine eigene Fehlerquelle darstellten.

KOLLOQUIUMSFRAGE

„**Verzerrt der Systemwechsel Ihre Daten?**“ — Auf der Ebene der Rohzahlen ja, deutlich. Deshalb wird daraus kein absoluter Wert, sondern ein *relativer* Index gebildet. Wie das den Bruch neutralisiert, steht auf der nächsten Folie.

kriminalitaetsindex(nb_per_year)

Das einzige monatlich variierende Merkmal — und die zweite Leakage-Sperre.

EIN monatliche Deliktzahlen, Einwohnerzahlen, Fensterlänge

AUS Indexspalte je Stadtteil-Monat, dazu crime_rate_raw

DEFINITION — LOCATION QUOTIENT

$$\text{rate}(i, t) = \text{Delikte}(i, \text{Fenster bis } t-1) / \text{Einwohner}(i)$$
$$\text{index}(i, t) = \text{rate}(i, t) / \text{rate}(\text{Stadt}, t)$$

1,0 heißt: Belastung wie im Stadtdurchschnitt *desselben* Monats.

WARUM RELATIV STATT ABSOLUT

- Der Systemwechsel 2018 verschiebt das stadtweite Niveau.
- Ein multiplikativer Sprung wirkt auf Zähler **und** Nenner und kürzt sich heraus.
- Verbleibende Limitation: Verschiebt sich die **Zusammensetzung** der erfassten Delikte ungleich über die Stadtteile, kürzt sie sich nicht heraus.

LEAKAGE - SPERRE

Das Zwölfmonatsfenster endet strikt im **Vormonat**. Der Index eines Monats enthält keine einzige Straftat dieses Monats.

NICHT VERWECHSELN

`crime_rate_raw` steht daneben, ist aber **kein Modellmerkmal** — nur Deskription für Kapitel 5.1. Sie enthält den Bruch von 2018.

berechne_quoten(df) und run_join()

Der Abschluss von Schritt 1.

BERECHNE_QUOTEN(DF)

EIN Zähler- und Nennerspalten

AUS Anteilswerte in [0,1]

- Nenner ≤ 0 ergibt `NaN` statt einer Division durch Null.
- Kriminalität taucht hier **nicht** auf — sie geht als relativer Index ein, nicht als Anteil.

RUN_JOIN()

EIN die Dateien aus `run_download()`

AUS `data/processed/einsaetze.parquet`

- Führt die Reihenfolge: SFFD bereinigen → ACS anfügen → Kriminalitätsindex → Bausubstanz → Quoten.
- Prüft am Ende die Vollständigkeit je Spalte gegen `VOLLSTAENDIGKEITS_SCHWELLE`.

ZWISCHENSTAND NACH SCHRITT 1

`einsaetze.parquet` enthält eine Zeile je Einsatz mit allen zehn Stadtteilmerkmalen. Auf dieser Ebene wird **nicht** modelliert — sie ist die Vorstufe für die Verdichtung in `s2`.

DATEI

s2_datensaetze.py

Verdichten und Zuschneiden. Hier entstehen die beiden Panels, die alle späteren Skripte lesen.

459

Zeilen

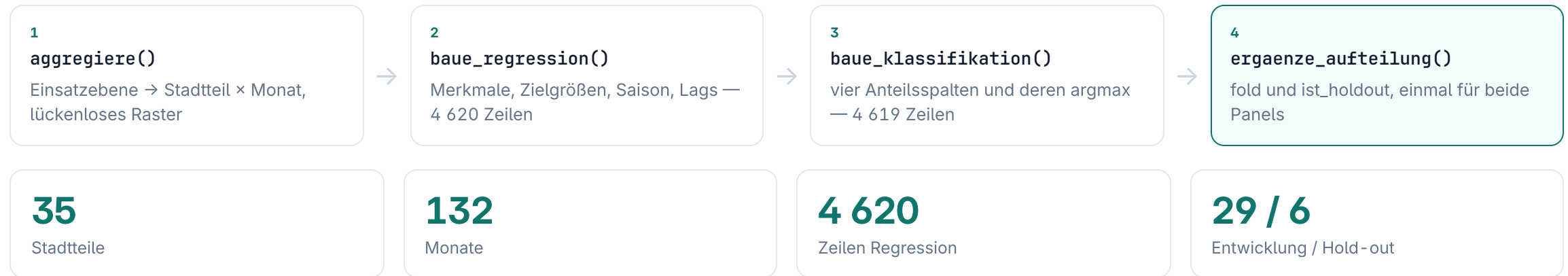
9

Funktionen

4 620

Zeilen im Panel

Der Weg zu den beiden Panels



DIE FAIRNESS-REGEL

Die Fold-Zuteilung wird **einmal** berechnet und auf **beide** Panels angewandt. Nur so sehen Mengen- und Strukturstrang dieselben Stadtteile im Test — sonst wären die beiden Stränge nicht vergleichbar.

aggregiere(von, bis, ...)

Verdichtet die Einsatzebene auf Stadtteil × Monat.

EIN Zeitgrenzen als jahr_monat-Schlüssel, inklusive Lag-Vorlauf

AUS ein Panel mit einer Zeile je Stadtteil und Monat

WAS „VOLLSTÄNDIGES RASTER“ HEISST

Auch ein Stadtteil-Monat **ohne** Einsätze bekommt eine Zeile — mit dem Wert Null. Ohne dieses Raster fehlte ein ruhiger Monat stillschweigend.

WARUM DAS ENTSCHEIDEND IST

- Die Lags arbeiten mit `shift()` über die Zeilenfolge.
- Fehlt eine Zeile, verrutscht `lag_1` auf den vorletzten statt den letzten Monat — **ohne Fehlermeldung**.
- Parkgebiete ohne Wohnbevölkerung werden gesondert behandelt.

KOLLOQUIUMSFRAGE

„**Wie viele Nullmonate gibt es?**“ — In der Praxis sehr wenige; die Einsatzzahlen liegen zwischen 6 und 280 je Stadtteil-Monat. Das Raster ist trotzdem konstruktiv nötig, weil man sich auf „kommt nicht vor“ nicht verlassen darf.

baue_regression(vorlauf, verbose)

Der Mengenstrang: wie viele Einsätze fallen an?

EIN einsaetze.parquet und die Zahl der Vorlaufmonate

AUS 4 620 Zeilen × 25 Spalten

WAS IN DEN DATENSATZ KOMMT

- **10 Prädiktoren** — sozioökonomisch, kriminalitätsbezogen, baulich
- **2 Zielgrößen** — `anzahl_einsaetze` und `einsaetze_je_1000_ew`
- **Exposition** — `gesamtbevoelkerung`, Offset des Poisson-GLM
- **Saison** — `monat_sin`, `monat_cos`
- **Lags** — bleiben im Datensatz, sind aber kein Modellmerkmal

DER LAG - VORLAUF

Aggregiert wird ab `START` minus zwölf Monaten, damit `lag_12` schon für den ersten Analysemonat definiert ist. Danach wird auf `START` zugeschnitten.

WARUM SAISON ALS SIN/COS

Der Monat als Zahl gäbe Dezember und Januar den Abstand 11 statt 1. Über Sinus und Kosinus wird das Jahr zum Kreis.

baue_klassifikation(regression, verbose)

Der Strukturstrang: welche Art von Einsätzen dominiert?

EIN der fertige Regressionsdatensatz

AUS 4 619 Zeilen × 29 Spalten mit dominante_einsatzart

WIE DIE ZIELGRÖSSE ENTSTEHT

- Die NFIRS-Codes werden auf **vier Klassen** abgebildet: Brand, Rettung/EMS, Technische Hilfe, Fehlalarm.
- Je Stadtteil-Monat wird der **Anteil** jeder Klasse berechnet.
- dominante_einsatzart ist der **argmax** über diese vier Anteile.

WARUM NICHT DER EINZELNE EINSATZ?

Innerhalb eines Stadtteil-Monats tragen alle Einsätze **identische** Strukturmerkmale. Auf Einzelfallebene war deshalb nichts zu holen: 49,9 % Treffer gegen 48,2 % für bloßes Raten. Zielgröße ist die **Zusammensetzung** der Einsatzlast.

DIE FOLGE, DIE SPÄTER ZURÜCKKOMMT

Ein argmax über vier Anteile ist **kein beobachtetes Merkmal**. Liegen zwei Anteile dicht beieinander, entscheidet der Zufall der Monatsziehung. Genau das beziffert später `vorpruefung/v4_decke.py` als Decke A.

ergaenze_aufteilung(daten, versatz, selten)

Die dritte Leakage-Sperre — und die wichtigste Funktion des Ordners.

EIN Datensatz, ein Versatz für wiederholte Splits, die Zahl brand-dominierter Monate je Stadtteil

AUS derselbe Datensatz mit den Spalten fold und ist_holdout

DAS VERFAHREN

- Die 35 Stadtteile werden sortiert und reihum auf **sechs** Gruppen verteilt.
- **Gruppe 0 ist das Hold-out** — sechs Stadtteile, die bis zur Schlussbewertung unberührt bleiben.
- Gruppen 1 bis 5 sind die Folds der Kreuzvalidierung.
- Ein Stadtteil steht mit **allen** 132 Monaten in genau einer Gruppe.

DOPPELT STRATIFIZIERT

- **Erst** nach der Zahl brand-dominierter Monate — sonst hat ein Fold keinen Brand-Testfall und Macro-F1 mittelt über eine fehlende Klasse.
- **Dann** nach Bevölkerung — sonst wäre die Fold-Streuung nur ein Größeneffekt.

WARUM DAS KEIN LEAKAGE IST

Festgelegt wird ausschließlich, *welche Stadtteile gemeinsam getestet werden* — genau wie bei `StratifiedGroupKFold`. Kein Modell sieht dadurch eine Zeile mehr. Der Unterschied zum Zeitschnitt: Getestet wird auf **unbekannten Stadtteilen**, nicht auf einer unbekannten Zukunft.

Die vier kleinen Funktionen

Zwei Helfer und zwei Auskunftsfunktionen.

FOLD_MASKEN(DATEN, K)

EIN Datensatz mit fold-Spalte, Foldnummer k

AUS zwei boolesche Masken (Training, Test)

- Test = die Stadtteile dieses Folds mit allen Monaten.
- Training = alle übrigen Entwicklungsstadtteile, **ohne** Hold-out.
- Liest nur die Spalten der Datei — die Aufteilung wird nirgends neu erfunden.

BESCHREIBE_SPLITS(DATEN)

EIN Datensatz mit Fold-Spalten

AUS nichts, reine Konsolenausgabe

- Zeigt, welcher Stadtteil in welchem Fold getestet wird.
- Belegt, dass jeder Fold den **vollen Zeitraum** abdeckt — der sichtbare Unterschied zum Zeitschnitt.
- Liefert die Zahlen für Kapitel 5.2 und 5.4.

_SETZE_DATENTYPEN(D, MERKMALE)

- Vereinheitlicht auf NumPy-Typen: Merkmale `float64`, Schlüssel `int64`.
- Nötig, weil **eine einzige** nullable `Int64`-Spalte genügt, damit `X.to_numpy()` ein Objekt-Array liefert. `sklearn` fängt das still ab, `XGBoost` lehnt es ab.

_MONAT_MINUS(JAHR_MONAT, MONATE)

- Verschiebt einen Schlüssel wie `202403` um n Monate zurück.
- Nötig, weil `jahr_monat` eine Ganzzahl ist und `202401 - 1` nicht `202312` ergibt.

run(verbose)

Der Einstiegspunkt: baut beide Panels und schreibt sie.

EIN einsaetze.parquet

AUS regression.parquet und klassifikation.parquet

DIE REIHENFOLGE

- `baue_regression()`
- `baue_klassifikation()` — nimmt Zeilen, Zeitraum, Merkmale und Folds aus dem Regressionsdatensatz
- `ergaenze_aufteilung()` **einmal**, angewandt auf beide
- `beschreibe_splits()` zur Kontrolle

WARUM DIE ABLEITUNG SO HERUM LÄUFT

Der Klassifikationsdatensatz wird **aus** dem Regressionsdatensatz gebaut, nicht unabhängig. Damit ist konstruktiv ausgeschlossen, dass die beiden Stränge verschiedene Zeilen, Merkmale oder Folds sehen — es gibt keine zweite Stelle, an der das auseinanderlaufen könnte.

KOLLOQUIUMSFRAGE

„**Warum hat die Klassifikation eine Zeile weniger?**“ — Ein Stadtteil - Monat ohne Einsätze hat keine Anteile und damit keinen argmax. Diese Zeile wird ausgeschlossen statt einer Klasse zugewiesen; eine erfundene Klasse wäre eine erfundene Beobachtung.

DATEI

build.py

Der eine Startbefehl. Enthält keine Analyselogik — nur Ablaufsteuerung und Kontrollausgabe.

106

Zeilen

3

Funktionen

1

Befehl

Ablaufsteuerung und Selbstauskunft

SCHRITT(NUMMER, TITEL)

- EIN** Nummer wie „1/2“ und der Titel
- AUS** Startzeitpunkt für die Dauermessung
 - Gibt die Überschrift aus und startet die Uhr.

UEBERSICHT()

- EIN** die Konstante DATEIEN
- AUS** Konsolenausgabe: Zeilen, Spalten, Größe, Zeitraum
 - Steckbrief jeder erzeugten Datei.
 - Eine fehlende Datei wird als **FEHLT** gemeldet, statt den Lauf abubrechen — beim Teillauf ist so sofort sichtbar, was aussteht.

MAIN()

- EIN** optional das Argument „tests“
- AUS** Exitcode 0, bei „tests“ der Code des Testlaufs
 - Fährt s1, dann s2, dann die Übersicht.
 - Die Reihenfolge ist zwingend.

KOLLOQUIUMSFRAGE

„Was passiert, wenn ich `build.py` zweimal starte?“ — Die Downloads sind über die `DOWNLOAD_*` -Schalter abgeschaltet, es wird also nichts erneut geladen. Die Panels werden neu gebaut und überschrieben — bitgenau identisch, weil keine Zufallsgröße ohne festen Startwert beteiligt ist.

Was man über diesen Ordner wissen muss

Drei Sätze, die jede Rückfrage zur Datenbasis abdecken.

1 · DIE ANALYSEEINHEIT

Modelliert wird der **Stadtteil - Monat**, nicht der Einsatz. $35 \text{ Stadtteile} \times 132 \text{ Monate} = 4\,620$ Zeilen. Die Verdichtung ist eine bewusste Entscheidung, keine Bequemlichkeit — auf Einzelfallebene tragen alle Einsätze eines Monats dieselben Merkmale.

2 · DIE DREI SPERREN

Publikationsverzug beim ACS,
Rückwärtsfenster beim Kriminalitätsindex,
Stadtteiltrennung bei den Folds. Jede sitzt an genau einer Stelle und ist dort benannt.

3 · DIE STRUKTUR DER MERKMALE

Von zehn Prädiktoren variiert **einer** monatlich. Bausubstanz ist ein Snapshot 2020, Sozialdaten wechseln alle paar Jahre. Diese Eigenschaft der Datenbasis erklärt später den größten Teil der Ergebnisse.

DER SATZ, MIT DEM MAN DEN ORDNER VERTEIDIGT

„Die Aufbereitung legt fest, welche Zahl zu welchem Zeitpunkt sichtbar sein darf — und schreibt das Ergebnis in die Datei, statt es jedem Skript erneut zu überlassen.“ Deshalb stehen `fold` und `ist_holdout` als Spalten im Panel und nicht als Code im Modell.